Relatório — Analisador Léxico Pasquick e IDE Chronus

Oct 17, 2026 · @Someone

Identificação

Curso	Ciência da Computação
Disciplina	Compiladores
Professor	Rodrigo Freitas Silva
Trabalho	Trabalho 01 — Analisador Léxico
Aluno(s)	*Gustavo Wesley de Souza, Thiago da Silva Abreu Souza*
Linguagem	Pasquick (arquivos `.pqk`)
IDE	Chronus

Introdução

Este relatório descreve o primeiro trabalho da disciplina de Compiladores: um analisador léxico para um dialeto de Pascal, feito a partir da gramática do Anexo I, e uma pequena IDE para escrever e testar os programas.

Desde o começo a ideia foi fazer algo simples de entender e de explicar. Por isso usamos C puro, com a raylib só para a parte gráfica, e não usamos ferramentas como o Flex para gerar o analisador. O autômato foi escrito à mão, estado por estado. Deu mais trabalho, mas foi assim que entendemos de verdade como a análise léxica funciona por dentro.

A linguagem Pasquick

Batizamos a linguagem de Pasquick, uma mistura de Pascal com *quick*, porque no fundo ela é um Pascal reduzido ao que a gramática pede. A IDE ganhou o nome de Chronus, e os arquivos de código usam a extensão `.pqk`.

Na parte léxica, seguimos o que o enunciado define. Não há diferença entre maiúsculas e minúsculas, então `begin`, `BEGIN` e `Begin` são a mesma palavra. Identificadores começam com letra, depois podem ter letras, dígitos e `_`, e têm no máximo 15 caracteres. Palavras reservadas não podem ser usadas como nome de variável.

Uma particularidade é que os operadores lógicos são em português, `E` e `OU`, do jeito que aparecem na gramática. Isso tem um efeito curioso: não dá para ter uma variável chamada `e`. Também deixamos `FOR` e `TO` reservados desde já, porque o `for` vai entrar na análise sintática.

Os números podem ser inteiros (`10`) ou reais com ponto (`3.14`). Os literais ficam entre aspas simples e não podem quebrar linha. Comentários podem ser de uma linha, com `//`, ou de várias, entre `{ }` ou `(* *)`. Tanto os comentários quanto os espaços são descartados pelo analisador.

Um programa pequeno, só para dar uma ideia de como a linguagem fica:

```
program Exemplo;
var
  x : integer;
begin
  x := 0;
  while x < 10 do
    x := x + 1;
end.
```

Como foi o desenvolvimento

Antes de escrever qualquer código, lemos o enunciado e a gramática com calma, listamos todos os tokens e tentamos prever os casos estranhos. Algumas decisões saíram dessa etapa: literal não pode ocupar mais de uma linha, comentário dentro de comentário não é aceito, e algo como `12abc` é erro, e não um número seguido de um identificador.

A primeira versão do analisador não tinha interface nenhuma. Era um programa de console que lia um arquivo `.pqk` e imprimia os tokens, a tabela de símbolos e os erros. Parece um passo a mais, mas ajudou muito, porque deu para testar o autômato sozinho, sem misturar problema de interface com problema de análise.

Com o lexer funcionando, instalamos a raylib pelo instalador para Windows, que já vem com o gcc, e partimos para a IDE. O editor de texto foi feito do zero. O programa inteiro fica num único vetor de `char`, o cursor é só um índice nesse vetor, e inserir ou apagar um caractere é um `memmove`. É uma solução bem simples, mas para programas do tamanho

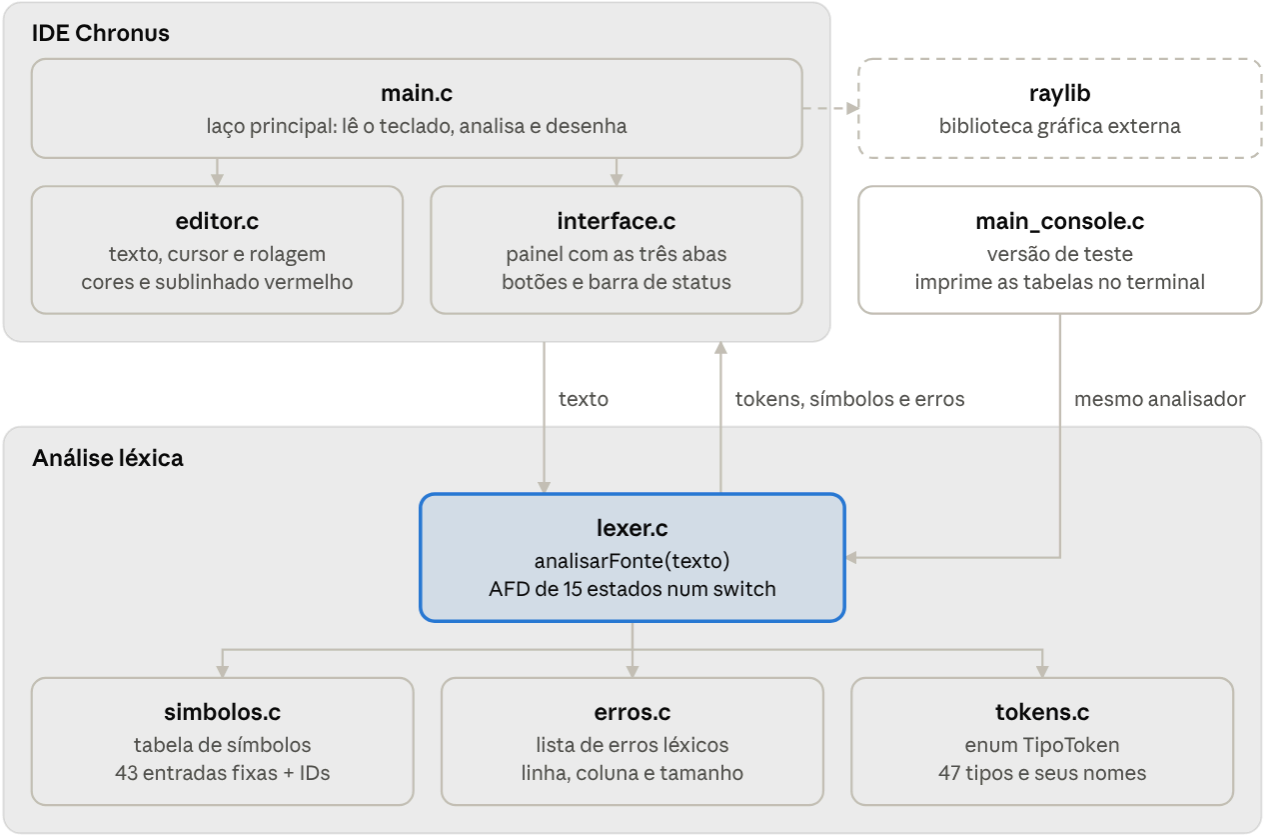
dos nossos exemplos funciona sem atraso nenhum. Por último ligamos as duas partes: a cada tecla o lexer roda de novo, e o resultado vira cores, sublinhados e as tabelas do painel.

Duas coisas pediram mais cuidado do que esperávamos. A primeira foram os acentos. O texto é UTF-8, e um caractere como `á` ocupa dois bytes, então se contássemos bytes a coluna dos erros sairia errada em qualquer linha com acento antes. Por isso as colunas são contadas por caractere, ignorando os bytes de continuação, tanto no lexer quanto no editor.

A segunda só apareceu quando vimos a IDE rodando. Um trecho com erro, como `12abc`, não vira token, e por isso o editor estava pintando ele com a mesma cor dos comentários. Ficava parecendo que o código estava comentado. Corrigimos pintando esses trechos de vermelho claro, usando a posição guardada em cada erro.

Diagrama dos módulos

O analisador léxico não depende da interface: IDE e console usam o mesmo código



Seta cheia: chamada ou dados. Tracejado: biblioteca externa, usada também por editor.c e interface.c.

módulos do programa · 8 arquivos .c + raylib

Organizamos o código de um jeito que o analisador léxico nem sabe que existe uma interface. Ele recebe um texto e devolve três coisas: a lista de tokens, a tabela de símbolos e a lista de erros. Foi isso que permitiu usar exatamente o mesmo `lexer.c` na IDE e na versão de console, sem mudar uma linha.

Como o analisador léxico funciona

Todo o trabalho acontece na função `analisarFonte()`, em `lexer.c`. Ela percorre o texto uma única vez, caractere por caractere, dentro de um laço com um `switch` sobre o estado atual. Cada `case` desse `switch` é um estado do autômato da próxima seção, então dá para ler o código com o desenho do lado.

No estado inicial, o analisador anota onde o lexema começa e decide para onde ir olhando só o caractere atual. Ao chegar num estado final, ele chama `emitir()`, que guarda o token com tipo, lexema, linha, coluna e atributo.

Um detalhe que simplificou bastante: palavras reservadas e identificadores seguem exatamente o mesmo caminho no autômato. Depois de ler a palavra inteira, o analisador passa ela para maiúsculo e procura na tabela de símbolos. Se ela já está lá como reservada, sai o token correspondente, como `WHILE`. Se não está, a palavra entra na tabela como identificador e sai um `ID`. Sem isso, teríamos que criar estados separados para cada uma das 24 palavras reservadas.

A tabela de símbolos é um vetor de structs com busca linear. Sabemos que não é a estrutura mais eficiente, mas com algumas dezenas de entradas a diferença não aparece, e o código fica fácil de acompanhar. Toda análise começa com 43 entradas fixas (24 palavras reservadas, 11 operadores e 8 sinais de pontuação), e cada identificador novo vai para o fim, junto com a linha onde apareceu pela primeira vez.

Erros e recuperação

Para a recuperação de erros escolhemos o modo pânico, que é a estratégia mais simples. Quando algo dá errado, o analisador registra a mensagem com linha, coluna e tamanho do trecho, joga fora a parte com problema e volta ao estado inicial. Assim um erro não esconde os próximos, e o usuário vê todos de uma vez. São cinco tipos de erro:

Erro	Exemplo	O que acontece
Caractere inválido	`idade@`, `#`, `_teste`, acento fora de string	o caractere é descartado
Identificador com mais de 15 caracteres	`nomeMuitoMaiorQueOPermitido`	o identificador inteiro é descartado

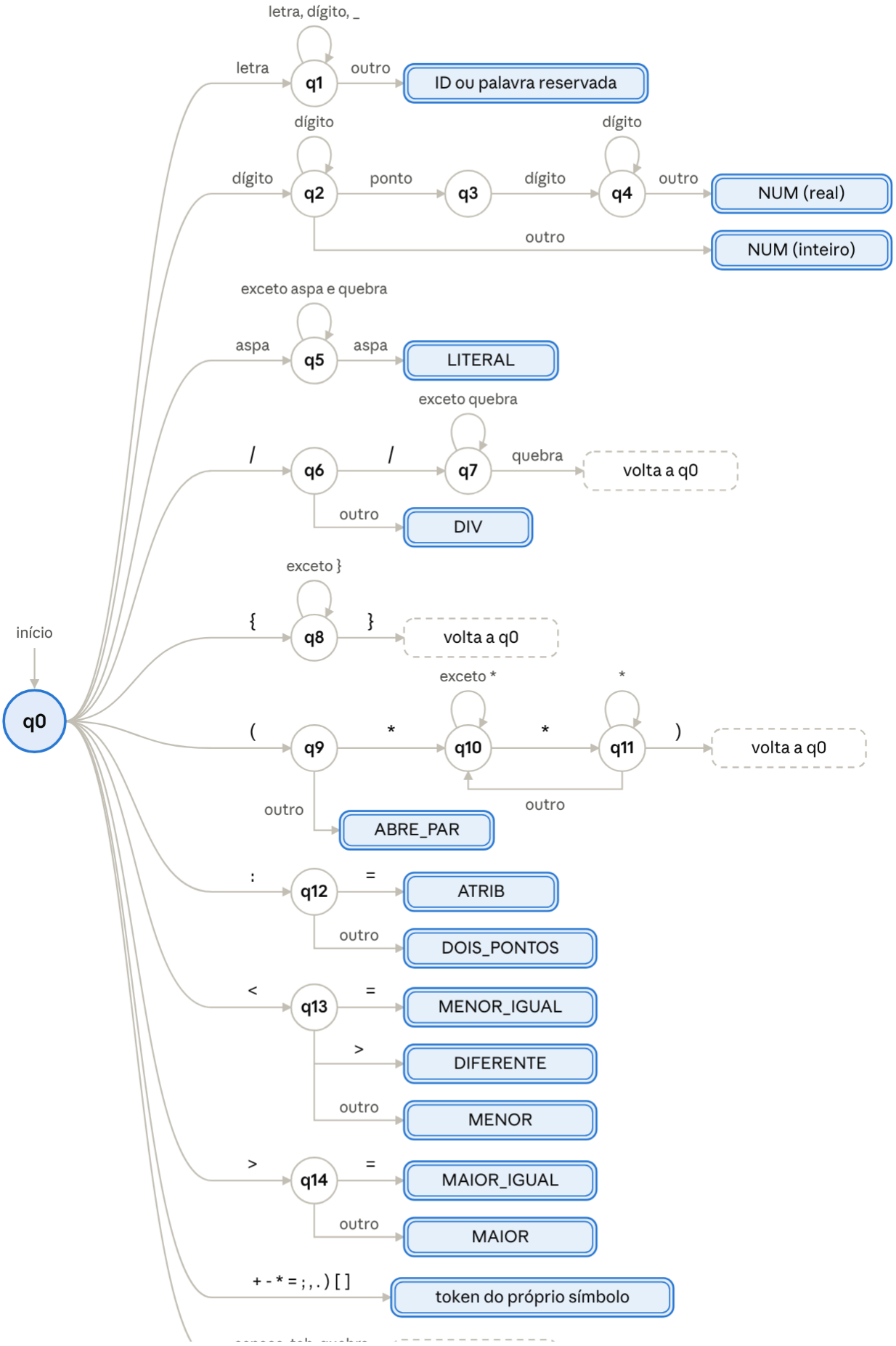
Erro	Exemplo	O que acontece
Número mal formado	`12abc`, `3.`	o trecho é descartado até o próximo separador
Literal não fechado	`'sem fechar`	o literal termina no fim da linha
Comentário não fechado	`{` ou `(*` sem fechamento	o erro é marcado onde o comentário abriu

Autômato finito determinístico

O autômato tem 15 estados, de q0 a q14. No desenho, cada ramo ocupa uma linha: o primeiro caractere do lexema escolhe o ramo, e dali em diante o analisador olha um caractere de cada vez para decidir se continua, se emite o token ou se descarta o que leu.

Autômato do analisador: o primeiro caractere escolhe o ramo

Cada linha corresponde a um ramo do switch em lexer.c





Círculo: estado. Borda dupla: estado final, emite o token. Tracejado: descarta o lexema e volta a q0. Transição não desenhada: erro léxico, tratado em modo pânico (volta a q0).

AFD do analisador léxico · 15 estados

A palavra "outro" aparece bastante no diagrama. Ela quer dizer qualquer caractere que não continua o lexema atual. Esse caractere não é consumido: o analisador emite o token, volta a q0 e começa o próximo lexema justamente por ele. A tabela abaixo liga cada estado ao nome que ele tem no código.

Estado	Nome em `lexer.c`	O que significa
q0	`Q0`	início de um novo lexema
q1	`Q_ID`	lendo identificador ou palavra reservada
q2	`Q_NUM`	lendo a parte inteira de um número
q3	`Q_NUM_PONTO`	leu o ponto, espera um dígito
q4	`Q_NUM_REAL`	lendo a parte decimal
q5	`Q_LITERAL`	dentro de um literal
q6	`Q_BARRA`	leu `/`: pode ser divisão ou comentário
q7	`Q_COMENT_LINHA`	dentro de um comentário `//`
q8	`Q_COMENT_CHAVE`	dentro de um comentário `{ }`
q9	`Q_ABRE_PAR`	leu `(`: pode ser parênteses ou comentário
q10	`Q_COMENT_PAR`	dentro de um comentário `(* *)`
q11	`Q_COMENT_PAR_AST`	no comentário `(* *)`, acabou de ler `*`
q12	`Q_DOIS_PONTOS`	leu `:`, pode virar `:=`
q13	`Q_MENOR`	leu `<`, pode virar `<=` ou `<>`
q14	`Q_MAIOR`	leu `>`, pode virar `>=`

Existe ainda um estado de erro no código, o `Q_NUM_ERRO`, que entra em ação quando um número aparece grudado em letras, como em `12abc`. Ele consome o resto da palavra e registra o erro. Deixamos ele fora do desenho para não poluir o diagrama, já que é só um caminho de erro.

Tabela de tokens

A tabela a seguir lista os 47 tokens da linguagem. A coluna Valor é o código numérico de cada token, que é simplesmente a posição dele no `enum TipoToken`, em `tokens.h`. A coluna Atributo mostra o que vai junto com o token quando ele é encontrado. Para palavras reservadas, operadores e pontuação é a posição fixa na tabela de símbolos (TS), já que eles são carregados antes da análise. As expressões regulares valem sem diferenciar maiúsculas de minúsculas.

Token	Lexema	Expressão regular	Atributo	Valor
PROGRAM	program	`program`	TS[0]	0
BEGIN	begin	`begin`	TS[1]	1
END	end	`end`	TS[2]	2
CONST	const	`const`	TS[3]	3
VAR	var	`var`	TS[4]	4
INTEGER	integer	`integer`	TS[5]	5
REAL	real	`real`	TS[6]	6
CHAR	char	`char`	TS[7]	7
STRING	string	`string`	TS[8]	8
PROCEDURE	procedure	`procedure`	TS[9]	9
FUNCTION	function	`function`	TS[10]	10
IF	if	`if`	TS[11]	11
THEN	then	`then`	TS[12]	12
ELSE	else	`else`	TS[13]	13
WHILE	while	`while`	TS[14]	14
DO	do	`do`	TS[15]	15
REPEAT	repeat	`repeat`	TS[16]	16
UNTIL	until	`until`	TS[17]	17
BREAK	break	`break`	TS[18]	18
CONTINUE	continue	`continue`	TS[19]	19
E	e	`e`	TS[20]	20

Token	Lexema	Expressão regular	Atributo	Valor
OU	ou	`ou`	TS[21]	21
FOR	for	`for`	TS[22]	22
TO	to	`to`	TS[23]	23
ID	contador, x, Media_1	`[A-Za-z][A-Za-z0-9_]{0,14}`	TS[43] em diante, na ordem em que aparecem	24
NUM	10, 3.14	`[0-9]+(\.[0-9]+)?`	inteiro ou real + valor numérico	25
LITERAL	'Pasquick!'	`'[^'\n]*'`	o próprio texto	26
ATRIB	:=	`:=`	TS[24]	27
MAIS	+	`\+`	TS[25]	28
MENOS	-	`-`	TS[26]	29
MULT	*	`\*`	TS[27]	30
DIV	/	`/`	TS[28]	31
IGUAL	=	`=`	TS[29]	32
DIFERENTE	<>	`<>`	TS[30]	33
MENOR	<	`<`	TS[31]	34
MENOR_IGUAL	<=	`<=`	TS[32]	35
MAIOR	>	`>`	TS[33]	36
MAIOR_IGUAL	>=	`>=`	TS[34]	37
PONTO_VIRGULA	;	`;`	TS[35]	38
VIRGULA	,	`,`	TS[36]	39
PONTO	.	`\.`	TS[37]	40
DOIS_PONTOS	:	`:`	TS[38]	41
ABRE_PAR	(	`\(`	TS[39]	42
FECHA_PAR	)	`\)`	TS[40]	43
ABRE_COL	[	`\[`	TS[41]	44
FECHA_COL	]	`\]`	TS[42]	45

Token	Lexema	Expressão regular	Atributo	Valor
EOF	(fim do arquivo)	—	—	46

Espaços e comentários também passam pelo autômato, mas não geram token nenhum. As expressões deles são `[ \t\r\n]+` para os espaços, `//[^\n]*` para o comentário de linha, `\{[^}]*\}` para o comentário com chaves e `\(\*` até `\*\)` para o comentário com parênteses e asterisco.

A IDE Chronus

A Chronus é uma janela só. Em cima ficam os botões Novo, Salvar e Compilar, no meio fica o editor, embaixo um painel com os resultados e, no rodapé, uma barra de status.

A parte de que mais gostamos é a análise enquanto o usuário digita. Toda vez que o texto muda, o lexer roda de novo sobre o programa inteiro. Pode parecer exagero, mas para programas desse tamanho isso leva muito menos que um quadro de tela, então o erro aparece no momento em que é digitado e some quando é corrigido. Foi o jeito mais simples que encontramos de atender ao pedido do enunciado de achar os erros enquanto o programa está sendo escrito.

Os erros aparecem de várias formas ao mesmo tempo. O trecho ganha um sublinhado ondulado vermelho, parecido com o do corretor ortográfico, e o número da linha fica vermelho. Passando o mouse por cima, aparece a mensagem com linha e coluna. A barra de status também fica vermelha e mostra o erro da linha onde está o cursor.

Como os tokens já estavam prontos, colorir o código saiu quase de graça: palavras reservadas em azul, números em verde-claro, literais em laranja e operadores em amarelo. Comentários não viram token e ficam na cor padrão, verde.

O painel de baixo tem três abas, Tokens, Símbolos e Erros. Clicar numa linha de qualquer uma delas leva o cursor até aquele ponto do código, o que ajuda bastante quando o programa é maior que a tela.

Para usar, F5 (ou o botão Compilar) roda a análise e abre a aba de erros, Ctrl+S salva e Ctrl+V cola. Para abrir um arquivo, basta arrastar o `.pqk` para dentro da janela ou passar o caminho ao abrir o programa, como em `chronus.exe exemplos\correto.pqk`. Deixamos de fora seleção de texto, copiar e desfazer, para o editor continuar simples.

Testes

Para testar, montamos dois programas, que ficam na pasta `exemplos/`, e rodamos os dois tanto na versão de console quanto na IDE.

O primeiro, `correto.pqk`, tenta usar um pouco de tudo: variáveis e constantes, uma função, `while`, `if` com `E`, os três tipos de comentário (dois deles com acento, de propósito) e palavras escritas ora em maiúsculo, ora em minúsculo. O analisador gerou 109 tokens, 52 entradas na tabela de símbolos (as 43 fixas e mais 9 identificadores) e nenhum erro. Também conferimos que `x` e `X` caíram na mesma entrada da tabela, a TS[44].

O segundo, `erros.pqk`, foi escrito para dar errado, com pelo menos um erro de cada tipo. Os oito erros foram encontrados, todos com a linha e a coluna certas, e a análise continuou depois de cada um:

Linha:coluna	Trecho	Mensagem
4:8	`idade@`	Caractere invalido '@'
5:3	`nomeMuitoMaiorQueOPermitido`	Identificador com mais de 15 caracteres
6:3	`_teste`	Caractere invalido '_'
8:12	`12abc`	Numero mal formado
9:12	`3.`	Numero mal formado: falta digito depois do ponto
10:11	`'sem fechar;`	Literal nao fechado
11:18	`#`	Caractere invalido '#'
12:3	`{ comentario que nunca fecha`	Comentario { nao foi fechado

Na IDE testamos também a análise ao vivo, apagando e corrigindo erros direto no editor. O sublinhado e a linha na aba de erros somem assim que o trecho fica certo.

Conclusão

O trabalho cobre o que o enunciado pede para esta fase. O analisador reconhece todos os tokens da gramática, ignora espaços e comentários, não diferencia maiúsculas de minúsculas, monta a tabela de símbolos e mostra os erros com linha e coluna, inclusive enquanto o programa é digitado.

O que mais ficou para nós foi ver a teoria virar código quase sem adaptação. Cada estado do autômato virou um `case` do `switch`, então quando precisávamos tratar um caso novo, como diferenciar `(` de `(*`, o caminho era acrescentar um estado no desenho e um `case` no código. Também ficou claro na prática por que a recuperação de erros importa: sem o modo pânico, um único `@` no começo do programa esconderia todos os erros seguintes.

Para a próxima etapa, a análise sintática, o lexer já deixa as coisas encaminhadas. A lista de tokens termina com `EOF`, cada token guarda linha e coluna para as mensagens de erro, e `FOR` e `TO` já estão reservados para quando o `for` entrar na gramática.